Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра систем штучного інтелекту



Звіт

про виконання лабораторних та практичних робіт блоку № 4

На тему: «Двовимірні Масиви. Вказівники та Посилання. Динамічні масиви. Робота з масивами та алгоритми обробки.»

з дисципліни: «Основи програмування»

до:

Лабораторної Роботи №4
Лабораторної Роботи №5
Алготестер Лабораторної Роботи №2
Алготестер Лабораторної Роботи №3
Практичних Робіт до блоку №4

Виконав:
Студент групи ШІ-12
Белаш Матвій Валерійович

2026

Тема роботи:

Двовимірні Масиви. Вказівники та Посилання. Динамічні масиви. Робота з масивами та алгоритми обробки.

Мета роботи:

Навчитися працювати з двовимірними масивами, використовуючи вказівники та посилання. Вміти ініціалізовувати динамічні масиви та оперувати ними за допомогою вказівників та посилань.

Теоретичні відомості:

1) Теоретичні відомості з переліком важливих тем:

- Тема №1: Класи пам'яті у C++
- Тема №2: Вступ до Масивів і Вказівників
- Тема №3: Вказівники та Посилання
- Тема №4: Двовимірні Масиви
- Тема №5: Динамічні Масиви
- Тема №6: Алгоритми обробки та робота з масивами

2) Індивідуальний план опрацювання теорії:

- Тема №1: Класи пам'яті у C++
    - Джерела Інформації:
        - Розгортка до четвертого епіка
        - Пари з ОП
        - Простори інтернету та ШІ
    - Що опрацьовано:
        - Ознайомлений з класами пам’яті. Здебільшого орієнтуюсь, що де лежить та що чому належить. Загалом знаю як оперувати пам’яттю.
    - Статус: Ознайомлений частково
    - Початок опрацювання теми: 20.10.2026
    - Завершення опрацювання теми: 01.11.2026

- Тема №2: Вступ до Масивів і Вказівників
    - Джерела Інформації:
        - Посилання на відеоурок
        - Пари з ОП
        - Розгортка до четвертого епіка
        - Простори інтернету та ШІ
        - Власна практика й досвід
        - Лабораторна робота №4
    - Що опрацьовано:
        - Ознайомлений з вказівниками та їх прямою належністю до масивів. Знаю в чому проблема статичних масивів, та що з цим можна зробити. Знаю про основи вказівників та посилань.
    - Статус: Ознайомлений
    - Початок опрацювання теми: 20.10.2026
    - Завершення опрацювання теми: 01.11.2026

- Тема №3: Вказівники та Посилання
  - Джерела Інформації:
    - Розгортка до четвертого епіка
    - Пари з ОП
    - Посилання на відеоурок
    - Власна практика й досвід
    - Простори інтернету та ШІ
    - Лабораторна робота №4

    Що опрацьовано:
    - Здебільшого ознайомлений з принципом роботи вказівників та посилань, та як ними орудувати. Знаю, як їх використовувати у функціях та складних алгоритмах. В загальному принцип арифметики вказівників.
  - Статус: Ознайомлений частково
  - Початок опрацювання теми: 20.10.2026
  - Завершення опрацювання теми: 02.11.2026

- Тема №4: Двовимірні Масиви
  - Джерела Інформації:
    - Пари з ОП
    - Посилання на відеоурок
    - Власна практика й досвід
    - Простори інтернету та ШІ
  - Що опрацьовано:
    - Знаю, що таке двовимірні масиви, як ними оперувати і передавати у функції. Знаю, як вони працюють та створювати динамічні.
  - Статус: Ознайомлений
  - Початок опрацювання теми: 27.10.2026
  - Завершення опрацювання теми: 02.11.2026

- Тема №5: Динамічні Масиви
  - Джерела Інформації:
    - Пари з ОП
    - Розгортка до четвертого епіка
    - Посилання на відеоурок
    - Простори інтернету та ШІ
  - Що опрацьовано:
    - Знаю, як виділяти динамічну пам’ять для роботи з великими або тяжкими у виконанні об’єктами. Загалом знаю, як оперувати new та delete. Ознайомлений, для чого це потрібно, та коли використовувати.
  - Статус: Ознайомлений частково
  - Початок опрацювання теми: 20.10.2026
  - Завершення опрацювання теми: 02.11.2026

- Тема №6: Алгоритми обробки та робота з масивами
    - Джерела Інформації:
        - Простори інтернету та ШІ
        - Розгортка до четвертого епіка
        - Пари з ОП
    - Що опрацьовано:
        - Попрактикувався з використанням статичних та динамічних масивів у різних алгоритмах.
        - Загалом знаю, як використовувати масиви у алгоритмах пошуку та сортування.
    - Статус: Ознайомлений частково
    - Початок опрацювання теми: 17.10.2026
    - Завершення опрацювання теми: 02.11.2026

Виконання роботи:

1. Опрацювання завдання та вимог до програм та середовища:

Завдання №1 Theory Education Activities

- Деталі завдання:
  Вивчити та опрацювати теоретичний матеріал для роботи з Епіком

Завдання №2 Requirements management (understand tasks) and design activities (draw flow diagram and estimate tasks)

- Деталі завдання:
  Ознайомитися зі всіма вимогами до завдань та зробити блок-схеми до потрібних завдань

Завдання №3 Lab# programming: Lab 4

- Варіант завдання: 22
- Деталі завдання:

22.

Реалізувати з використанням масиву стек (перший прийшов, останній пішов), для чого організувати додавання, знищення елементів з масиву і друк масиву після кожної операції.

Рисунок 1 – Умова Lab 4

Завдання №4 Lab# programming: Lab 5

- Варіант завдання: 22
- Деталі завдання:

Використовуючи функції, розв'язати зазначене у варіанті завдання. Масив повинен передаватися у функцію як параметр.

22. У двовимірному масиві знайти середнє арифметичне першого стовпця й кількість елементів у кожному з наступних стовпців, що перевищують середнє арифметичне попереднього стовпця.

Рисунок 2 – Умова Lab 5

Завдання №5 Lab# programming: Algotester Lab 2

- Варіант завдання: 1
- Деталі завдання:

Lab 2v1

Limits: 1 sec., 256 MiB

У вас є дорога, яка виглядає як N чисел.

Після того як ви по ній пройдете - вашу втому можна визначити як різницю максимального та мінімального елементу.

Ви хочете мінімізувати втому, але все що ви можете зробити - викинути одне число з дороги, тобто забрати його з масиву.

В результаті цієї дії, яку мінімальну втому ви можете отримати в кінці дороги?

Input

У першому рядку ціле число N - кількість чисел

У другому рядку масив r, який складається з N цілих чисел

Output

Єдине ціле число m - мінімальна втома, яку можна отримати

Рисунок 3 – Умова Algotester Lab 2

Завдання №6 Lab# programming: Algotester Lab 3

- Варіант завдання: 3
- Деталі завдання:

Lab 3v3

Limits: 1 sec., 256 MiB

Вам дана стрічка s.

Ваше завдання зробити компресію стрічки, тобто якщо якась буква йде більше одного разу підряд у стрічці замінити її на букву + кількість входжень підряд.

Input

У першому рядку стрічка S

Output

Стрічка $S_{compressed}$

Рисунок 4 – Умова Algotester Lab 3

Завдання №7 Practice# programming: Class Practice Task

- Деталі завдання:

Редактор монохромного зображення

Задача

Ви пишете простий редактор зображень у відтінках сірого. Зображення задається прямокутною таблицею чисел від 0 до 9, де 0 це чорний піксель, а 9 найсвітліший. Програма показує таку таблицю як малюнок із символів і дозволяє застосовувати до неї перетворення.

Розміри зображення заздалегідь невідомі: їх задає користувач під час роботи. Тому пам'ять під зображення виділяється динамічно, а після повороту на дев'яносто градусів висота і ширина міняються місцями, і масив доводиться виділяти заново.

Обмеження: сторонні бібліотеки використовувати заборонено. Ввід і вивід лише через cin та cout.

Рисунок 5 – Умова Class Practice Task

Завдання №8 Practice# programming: Self Practice Task 1

- Деталі завдання:

Допоможе чи заб'є?

Обмеження: 2 сек., 256 МіБ

До Тойлет-мена, відомого вам білоруського супергероя, дуже часто звертаються по допомогу різні люди. Проте, очевидно, усім мужній гігант допомогти не зможе — не вистачить часу. Та й не дуже хоче.

Саме тому, коли до нього приходить певне SMS-повідомлення з проханням про допомогу, він погодиться допомогти тоді й лише тоді, коли в цьому повідомленні знайдеться хоча б k входжень рядка `TOILET`, які не перетинаються.

За заданим повідомленням s, яке складається з великих латинських символів, виведіть `YES`, якщо Тойлет-мен погодиться допомогти людині, яка написала це повідомлення. У протилежному разі виведіть `NO`.

Вхідні дані

У першому рядку задано одне ціле число k — мінімальна кількість незалежних входжень рядка `TOILET`.

Другий рядок містить рядок s — повідомлення, надіслане Тойлет-мену. Рядок містить лише великі латинські символи.

Вихідні дані

У єдиному рядку виведіть `YES` або `NO` — відповідь на задачу.

Рисунок 6 – Умова Self Practice Task 1

Завдання №9 Practice# programming: Self Practice Task 2

- Деталі завдання:

Спекотні дні пінгвінів

Обмеження: 2 сек., 256 МіБ

Ви собі навіть уявити не можете, як же спекотно пінгвінам на Мадагаскарі. Щоб хоч трішки охолодитись, вони випивають безалкогольні коктейлі, однак і з цим проблемно — руки не пристосовані до такого способу життя. Вам потрібно допомогти визначити пінгвінам, чи зможуть вони випити коктейль, що лежить на столі.

Для простоти будемо вважати, що коктейль на столі — круг із діаметром l, в той час, як рот пінгвіна в будь-який момент часу — прямокутник із шириною w, сторони якого паралельні осям координат. В початковий момент часу рот пінгвіна закритий, тому прямокутник вироджений — його висота 0 (інакше кажучи, він є горизонтальним відрізком). Верхня щелепа пінгвіна може розкритись не більше ніж на u дюймів відносно початкової позиції, в той час, як нижня — на d дюймів.

Будемо вважати, що пінгвін може випити коктейль, якщо той повністю впишеться в його рот. Допоможіть пінгвіну за всіма заданими параметрами визначити, чи зможе він випити коктейль.

Вхідні дані

У єдиному рядку задано 4 натуральні числа l, w, u та d — діаметр коктейлю, ширина роту і на скільки дюймів щелепи можуть розкритися відповідно.

Вихідні дані

Якщо пінгвін зможе випити коктейль, виведіть рядок `Three times Sex on the Beach, please!`, в протилежному випадку — `Forget about the cocktails, man!`.

Рисунок 7 – Умова Self Practice Task 2

Завдання №10 Practice# programming: Self Practice Task 3

- Деталі завдання:

Літня школа

Обмеження: 2 сек., 256 МіБ

Одного разу до Ужгорода на літню школу з алгоритмічного програмування приїхали n студентів, що сформували k команд. Відомо, що кожна команда складається з одного, двох або трьох студентів.

Вам необхідно визначити, скільки студентів було в кожній із команд.

Вхідні дані

Єдиний рядок містить два цілих числа n та k — кількості студентів та команд.

Вихідні дані

У єдиному рядку виведіть k цілих чисел a_j через пробіл. Тут a_j — кількість студентів у j-тій команді.

Якщо існує більше одного розв'язку — виведіть будь-який.

Якщо розв'язку не існує — виведіть `Impossible`.

Рисунок 8 – Умова Self Practice Task 3

Завдання №11 Practice# programming: Self Practice Task 4

- Деталі завдання:

Повний чіназес

Обмеження: 2 сек., 512 МіБ

Зеник закінчив змагатись в ICPC, випустився з університету, і влаштувався на роботу. Інколи він заробляє гроші, а інколи витрачає їх. Наразі на рахунку Зеника 0 гривень.

Всього Зеник здійснив n дій, в i-ту дію він або заробив a_i гривень, або витратив a_i гривень.

Якщо в якийсь момент часу баланс Зеника був від'ємним, то він був у боргу. Інакше, на думку Зеника, відбувся повний чіназес. Допоможіть Зенику визначити, чи був його баланс від'ємним!

Вхідні дані

Перший рядок містить одне ціле число n. i-ий з наступних n рядків може мати один з наступних двох форматів:

- `earn` a_i, що означає, що Зеник заробив a_i гривень;
- `spend` a_i, що означає, що Зеник витратив a_i гривень.

Вихідні дані

Якщо Зеник не був у боргу протягом даних n дій, в єдиному рядку виведіть `chinazes`. Інакше, виведіть `debt`.

Рисунок 9 – Умова Self Practice Task 4

Завдання №12 Result Documentation Report and Outcomes Placement Activities (Docs and Programs on GitHub)

- Деталі завдання:
  Документація готових файлів та звіту до вивантаження на GitHub

Завдання №13 Results Evaluation and Release

- Деталі завдання:
  Підсумкова перевірка роботи та реліз

2. Дизайн та планована оцінка часу виконання завдань:

Програма №1 **Редактор монохромного зображення**

- Блок-схема:

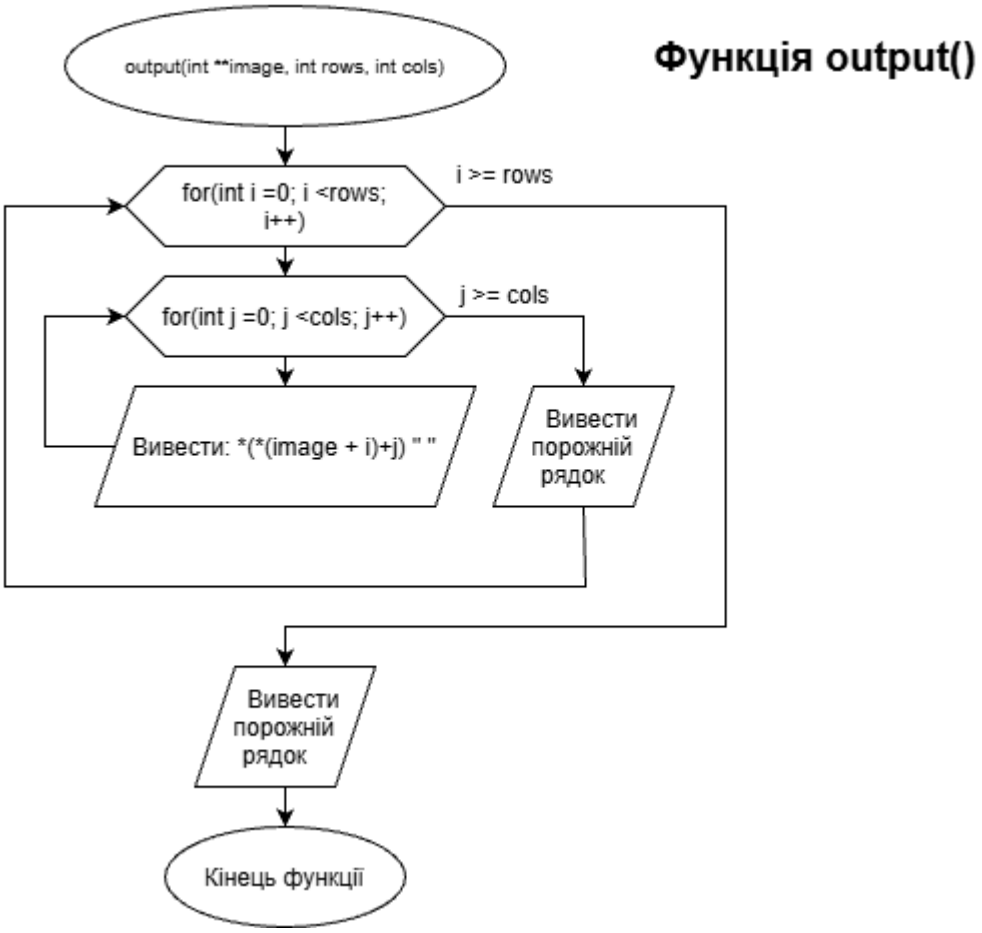


Рисунок 10 – Блок-схема функції output(): виведення матриці зображення

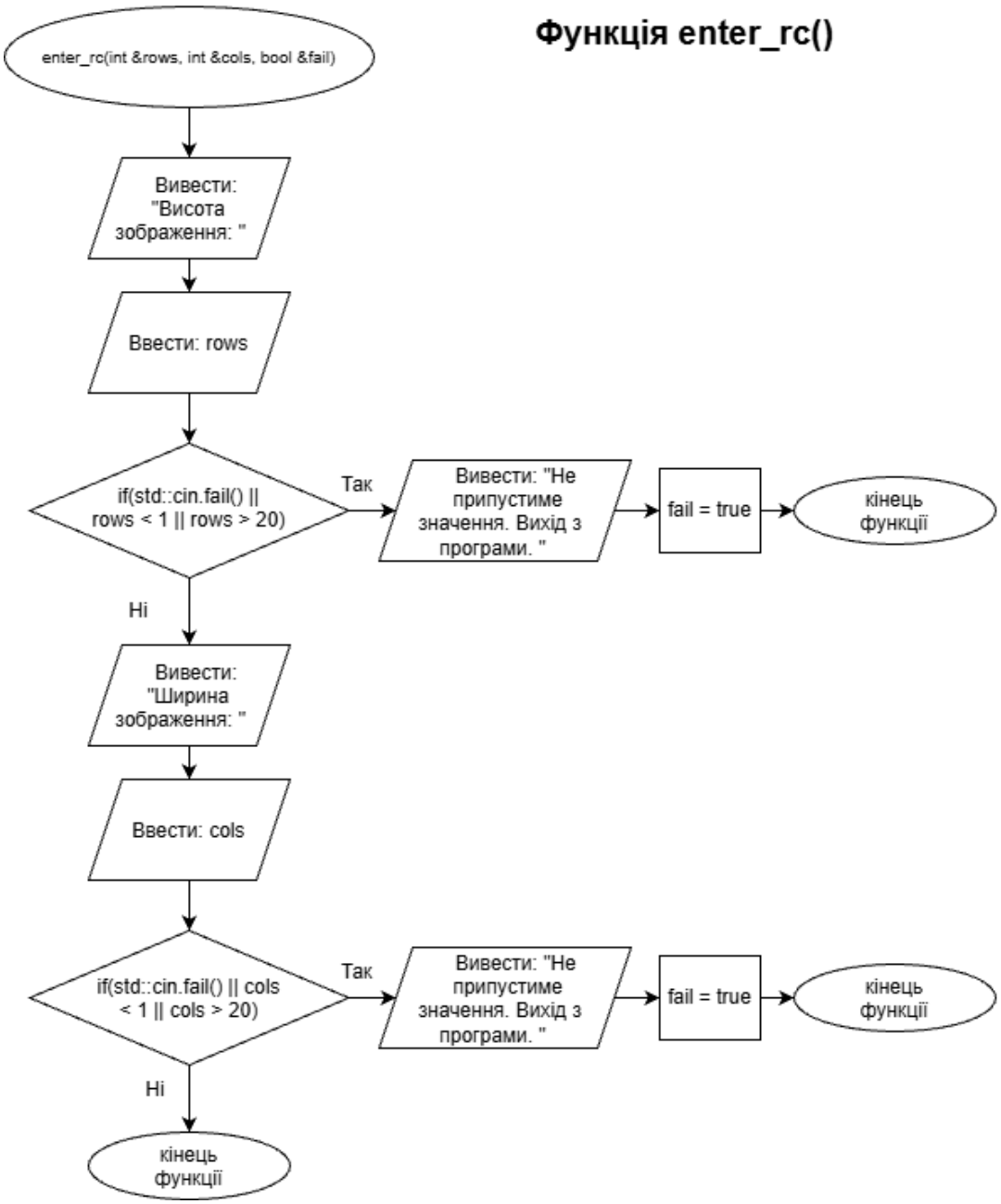


Рисунок 11 – Блок-схема функції enter_rc()

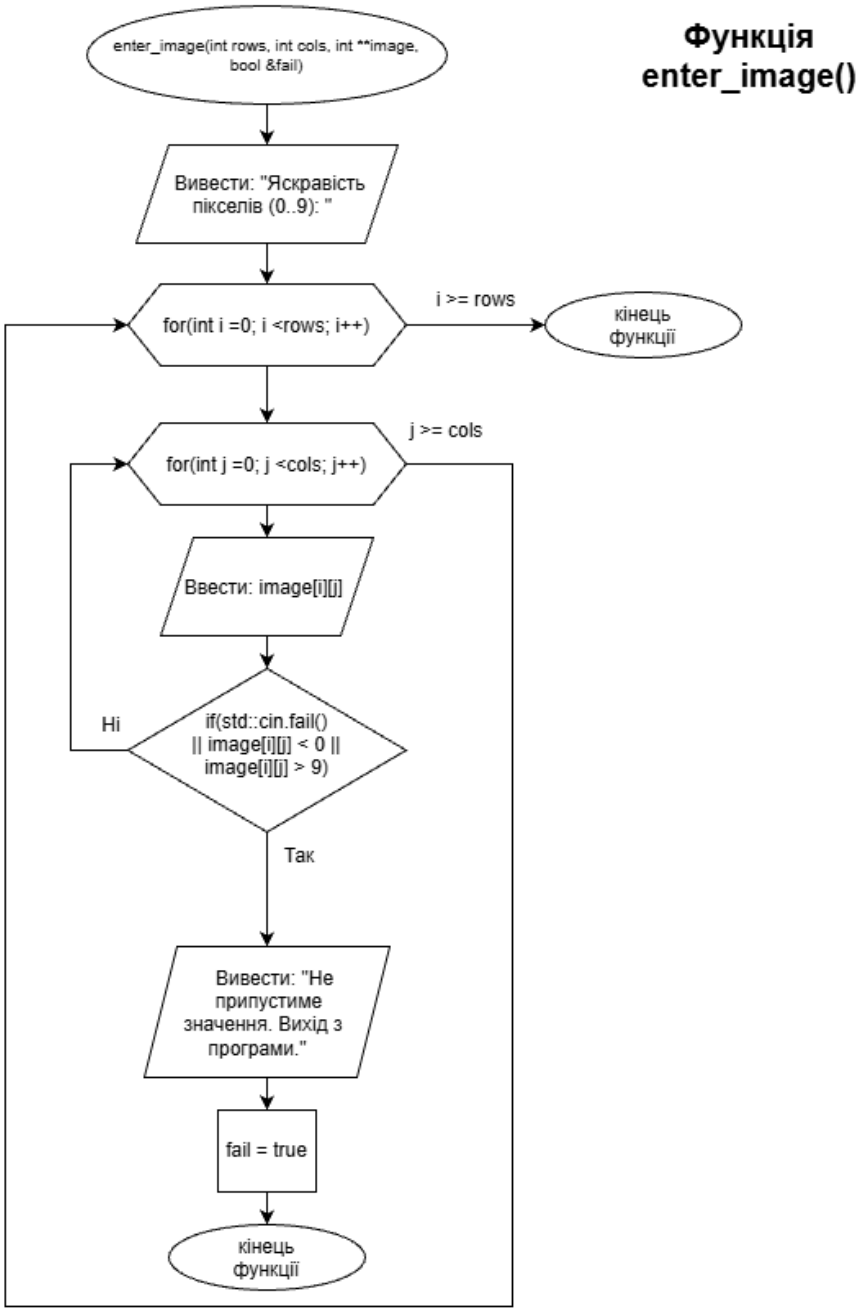


Рисунок 12 – Блок-схема функції enter_image()

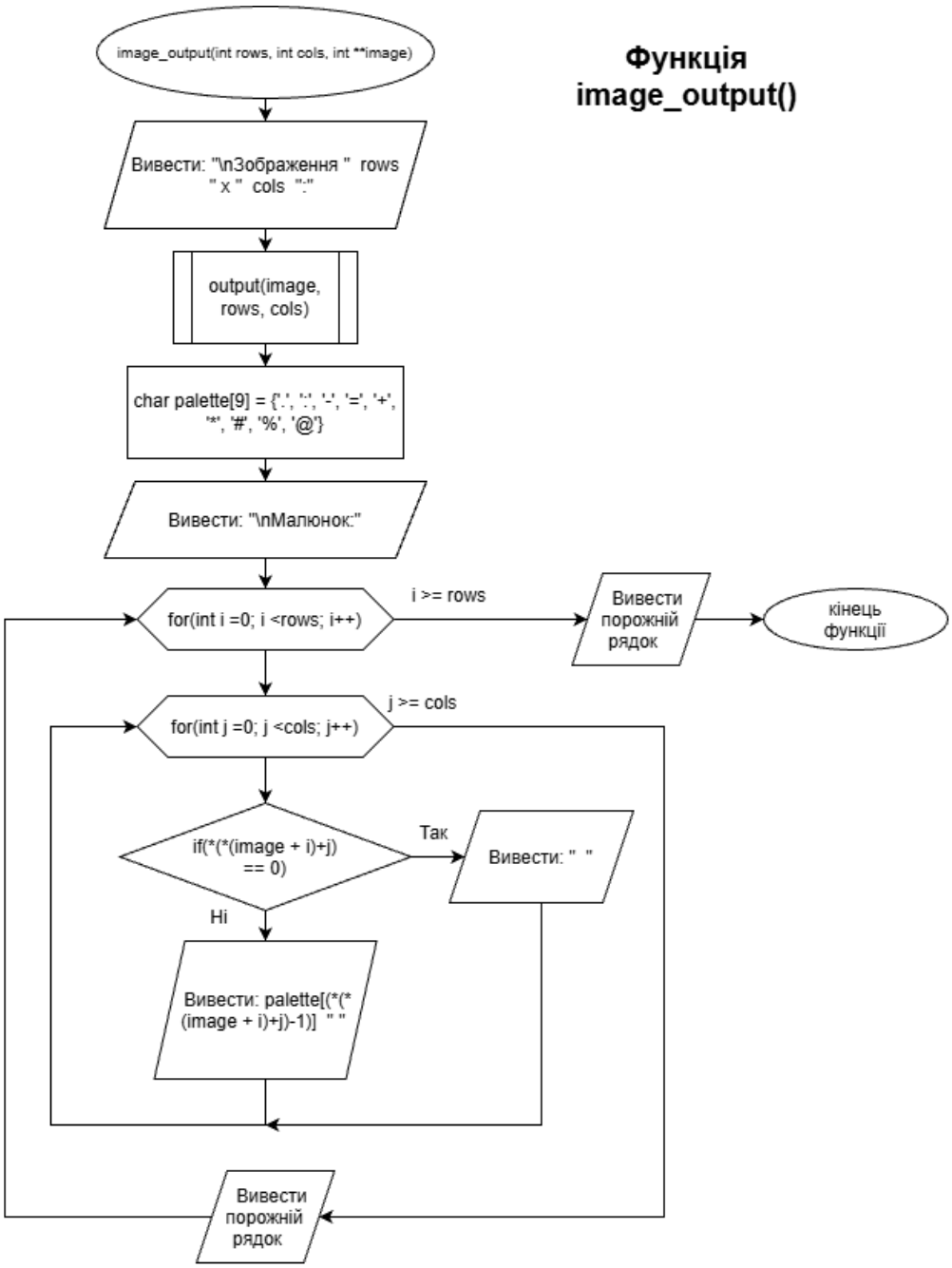


Рисунок 13 – Блок-схема функції image_output()

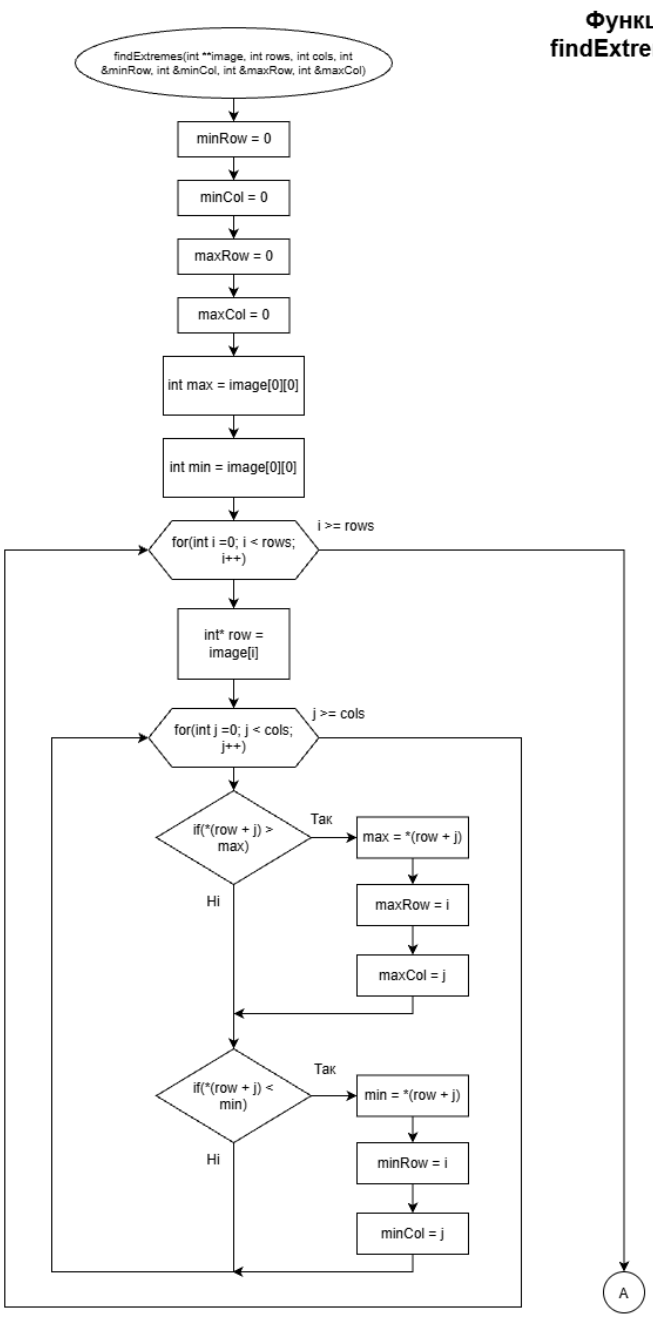


Рисунок 14 – Блок-схема функції findExtremes()

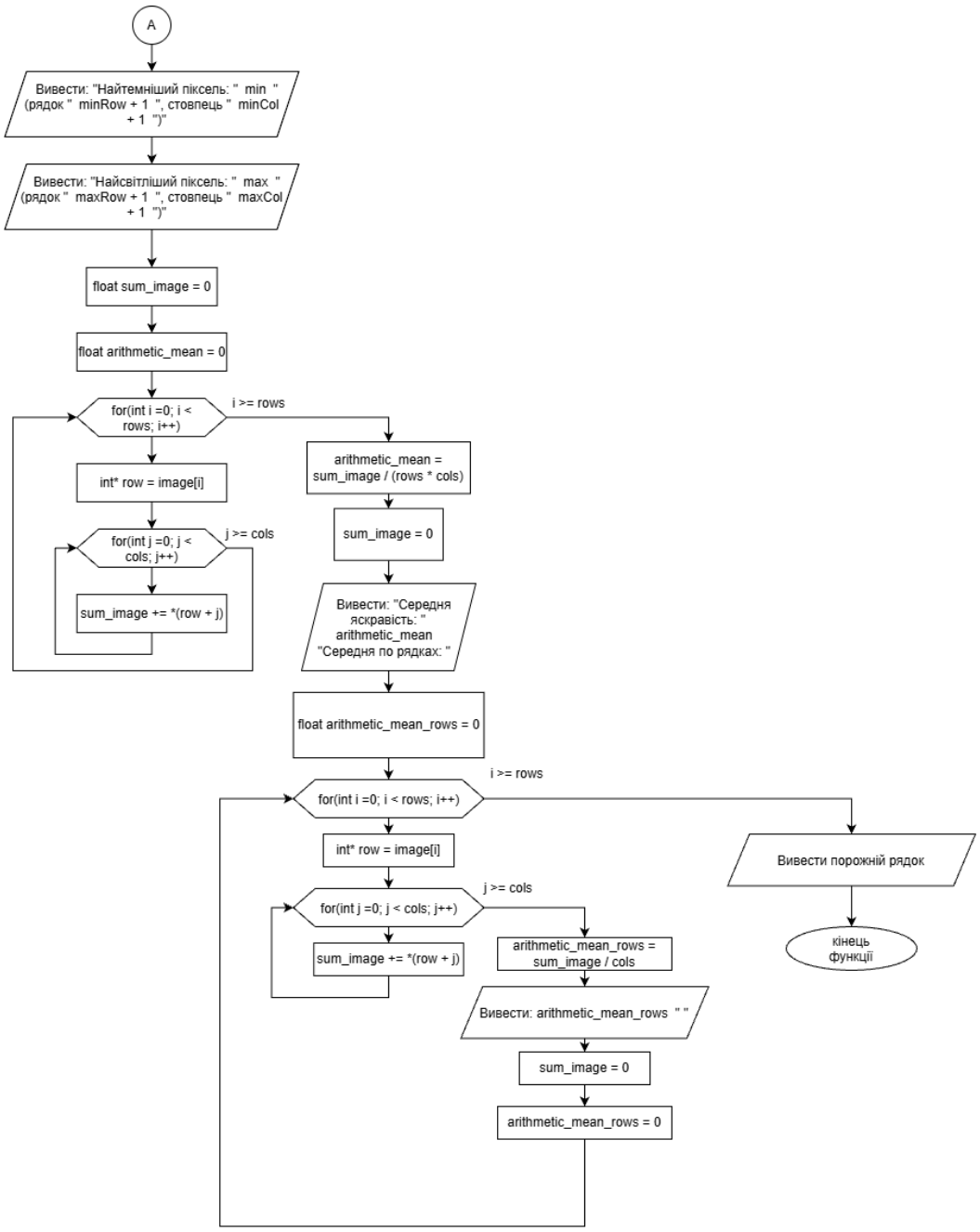


Рисунок 15 – Блок-схема функції findExtremes()

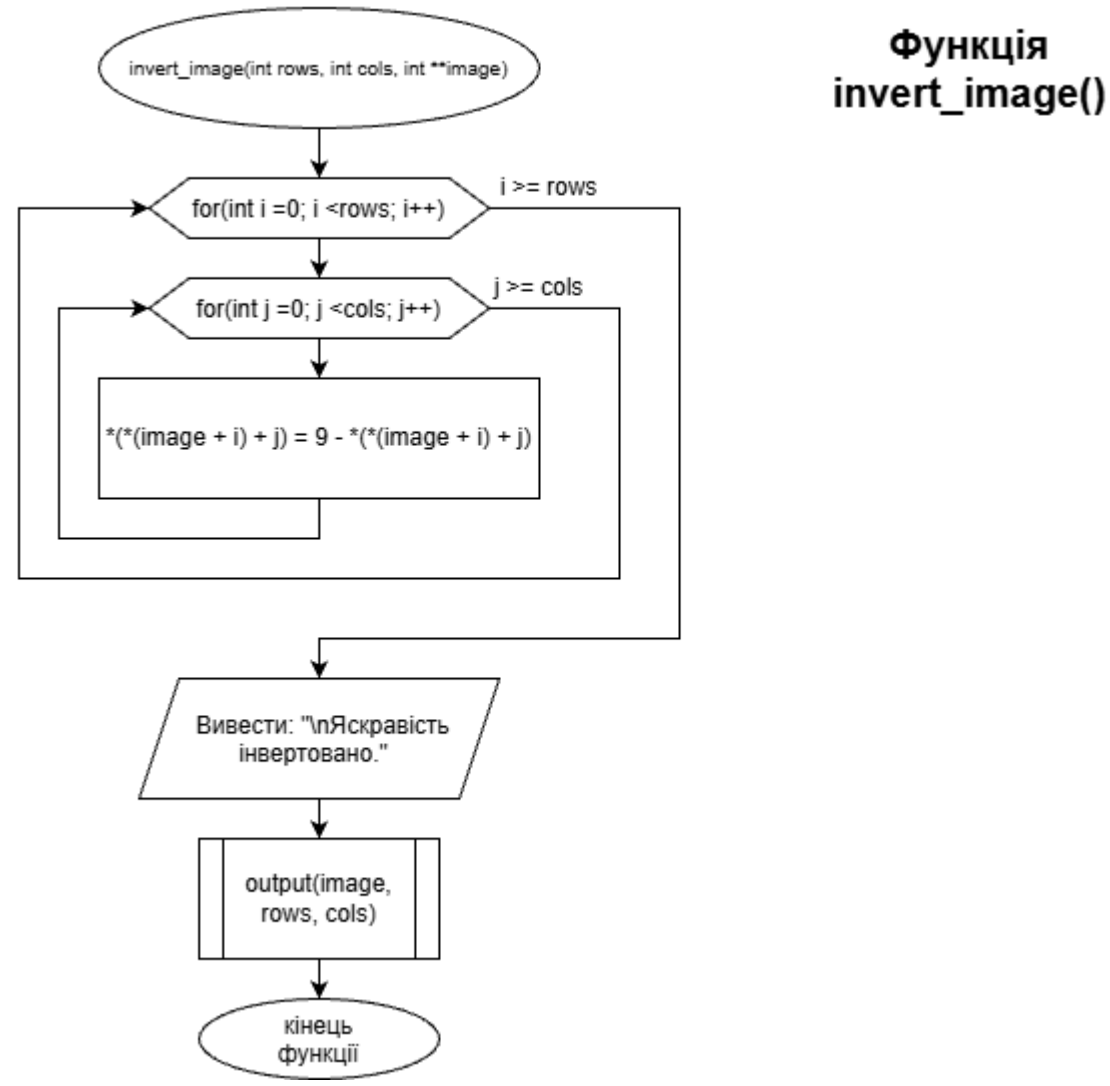


Рисунок 16 – Блок-схема функції invert_image()

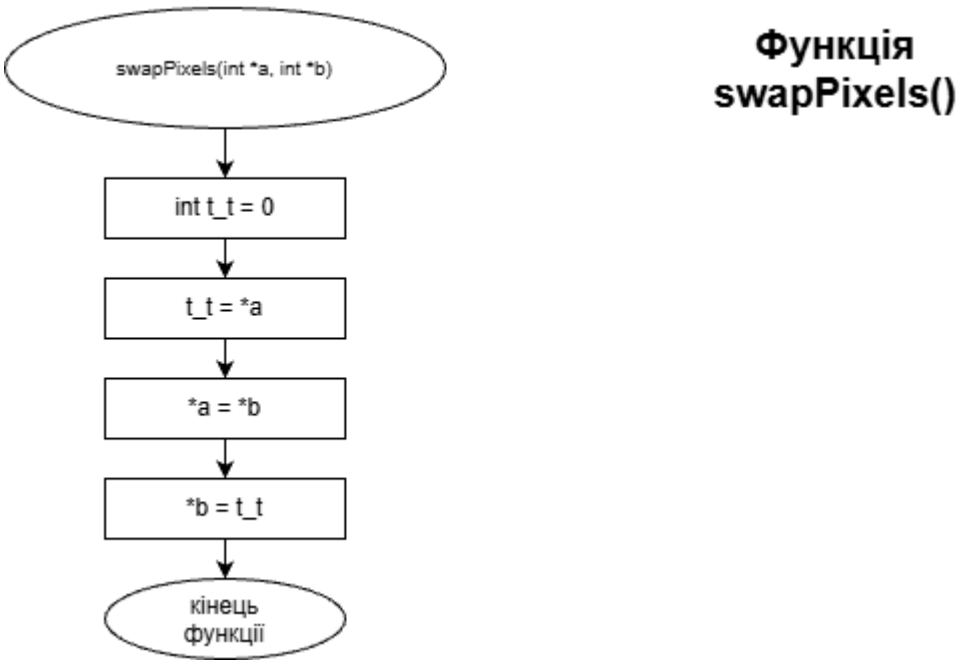


Рисунок 17 – Блок-схема функції swapPixels()

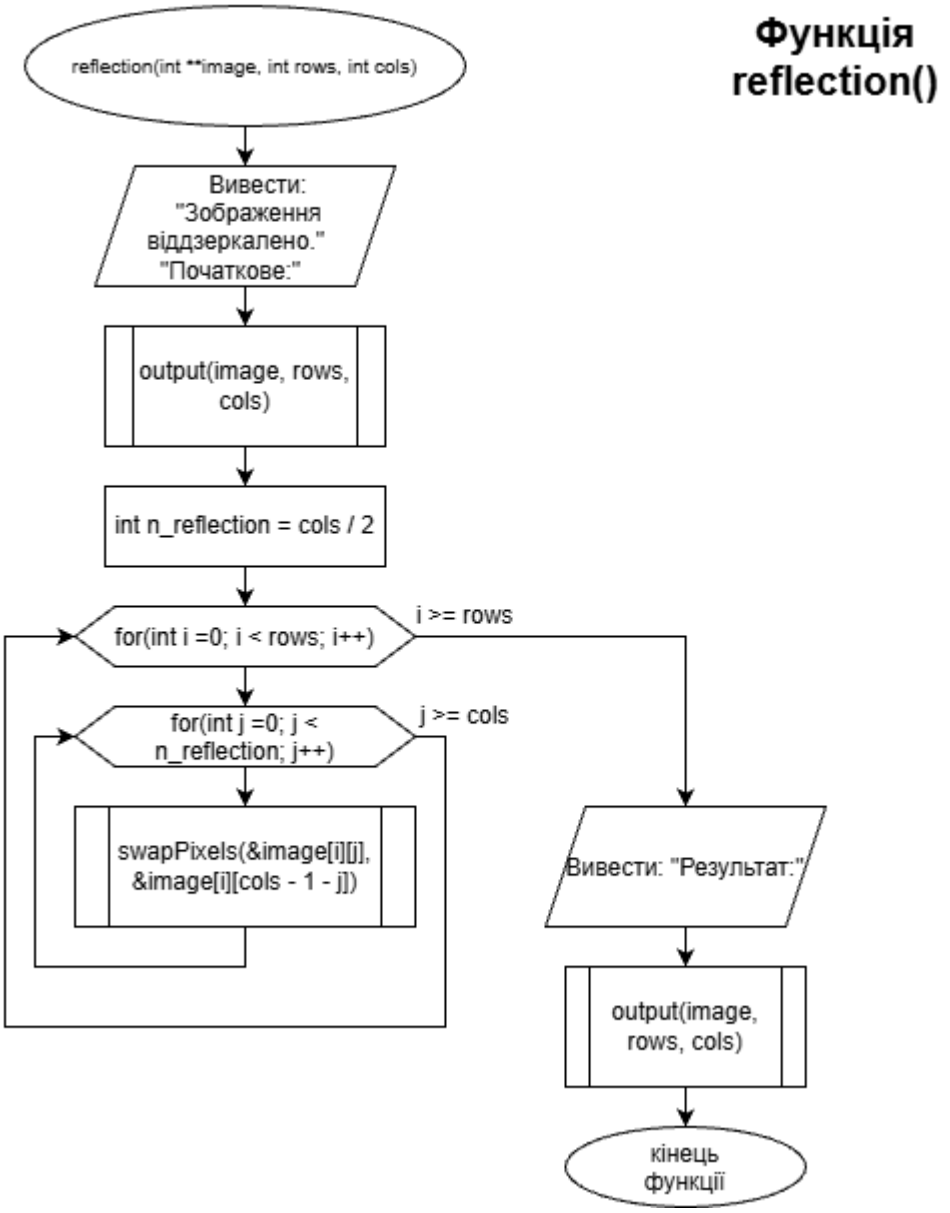


Рисунок 18 – Блок-схема функції reflection()

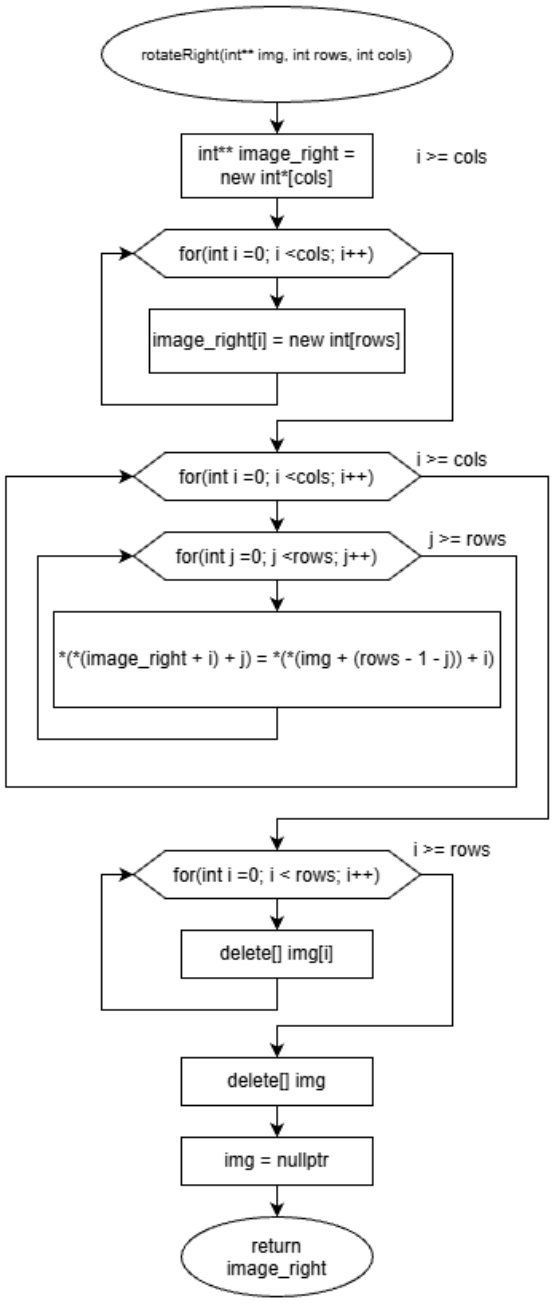


Рисунок 19 – Блок-схема функції rotateRight()

Функція Rotate_Output()

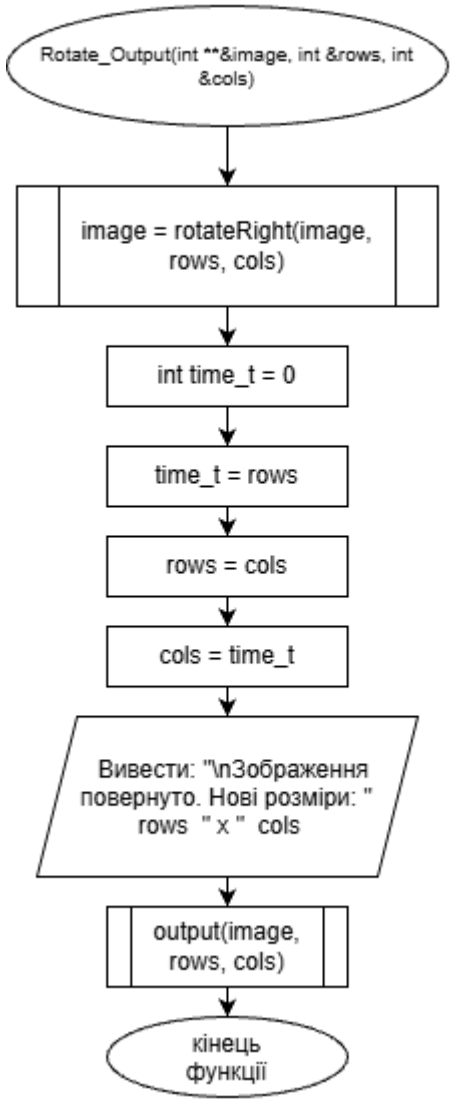


Рисунок 20 – Блок-схема функції Rotate_Output()

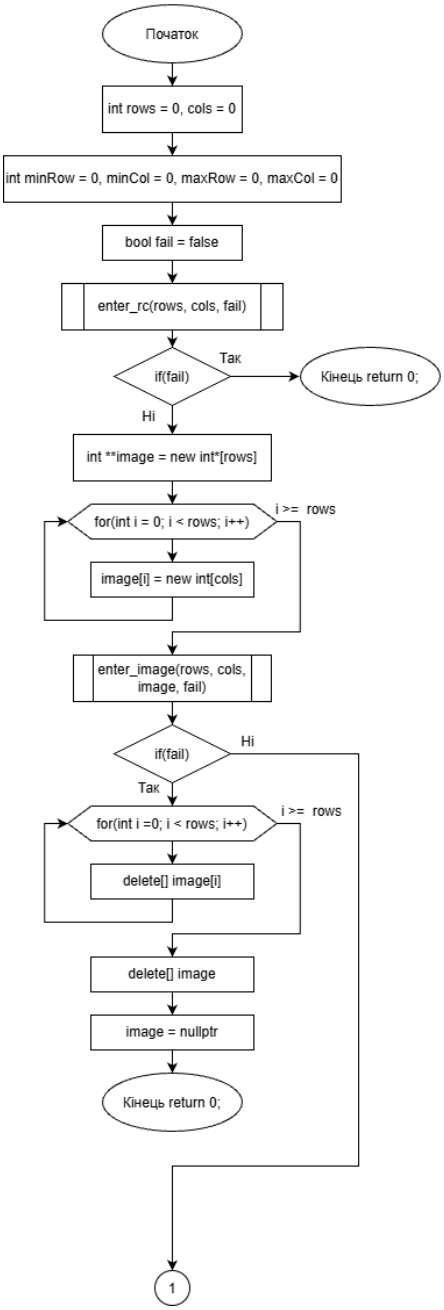


Рисунок 21 – Блок-схема функції main()

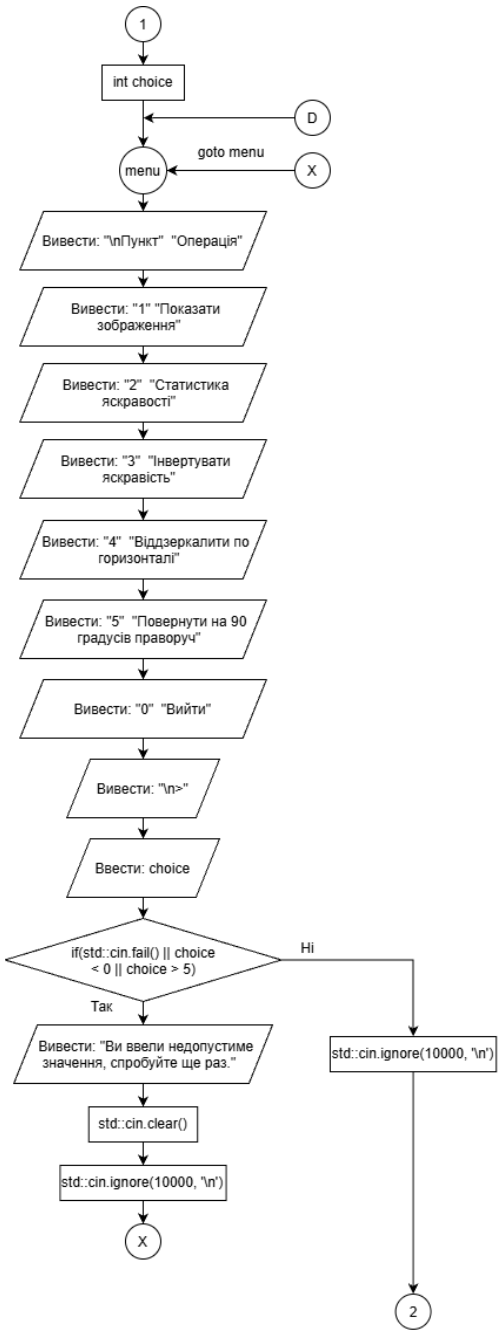


Рисунок 22 – Блок-схема функції main()

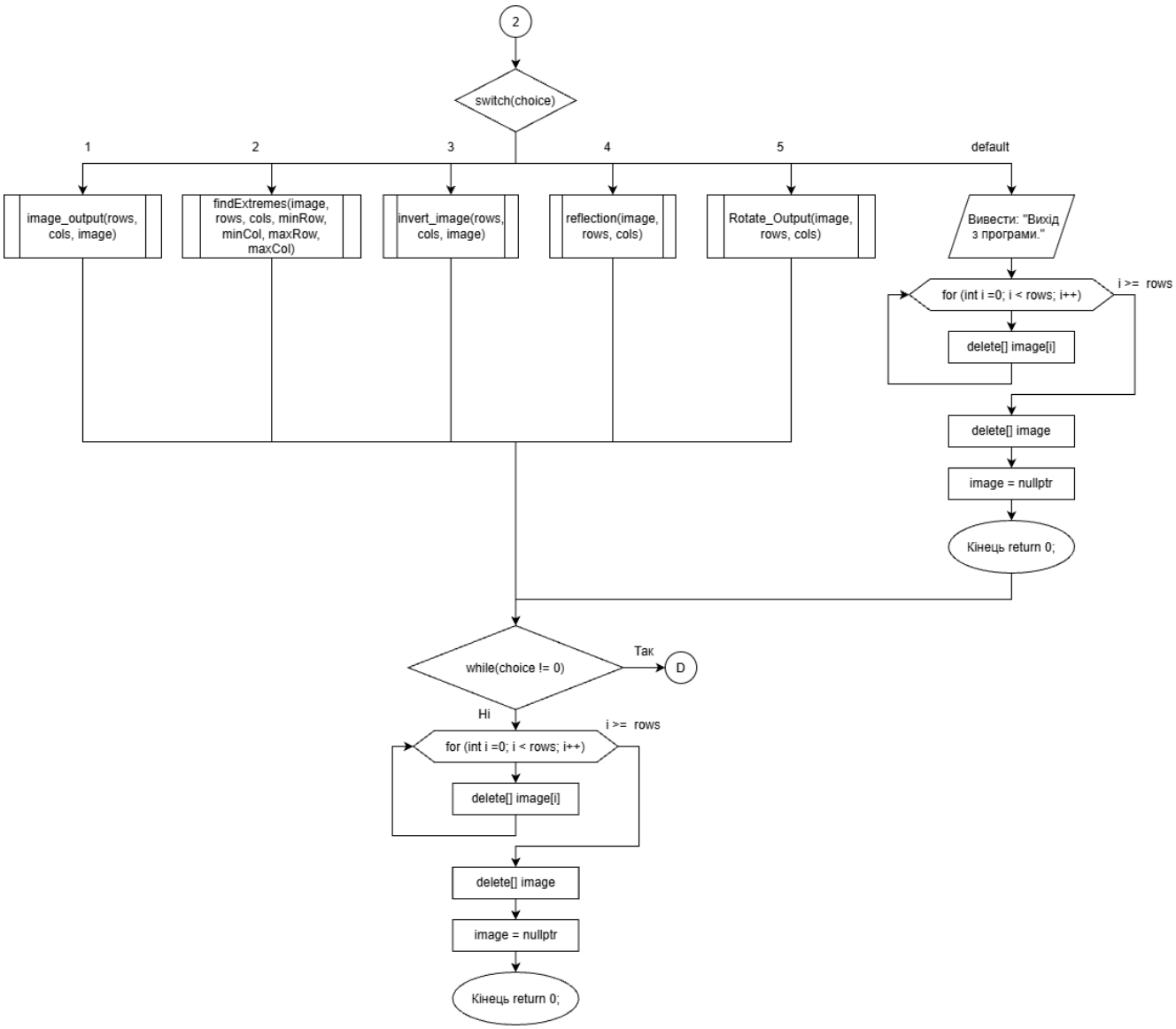


Рисунок 23 – Блок-схема функції main()

- Планований час на реалізацію: 3 години

3. Конфігурація середовища до виконання завдань:

Даний Епік не потребує конфігурацій

4. Код програм з посиланням на зовнішні ресурси:

Завдання №3 Lab# programming: Lab 4

```
#include <iostream>
using namespace std;


int main(){

    const int n_element = 100;
    int array_A[n_element];
    int n_moment = 0;
```

```
    int plus_minus = 0;
    char choice;

    start:

    cout << n_moment << "/" << n_element << " елементів масиву" << endl;

    plus:
    cout << "Скільки ви хочете додати елементів: ";   //вводимо скільки хочемо додати
    cin >> plus_minus;
    if(plus_minus > (n_element - n_moment)){
        cout << "Ви ввели забагато елементів. Додамо максимальну кількість елементів,
що ще лишилися для цього масиву" << endl;
        plus_minus = n_element - n_moment;
    }
    if(cin.fail() || plus_minus < 0){
        cout << "Ви ввели недопустиме значення. Спробуйте ще раз." << endl;
        cin.clear();
        cin.ignore(10000, '\n');
        goto plus;
    }
    if(plus_minus == 0){
        cout << "A = { ";
        for(int j =0; j <n_moment; j++){
            cout << *(array_A + j) << " ";
        }
        cout << "}" << endl;
    }

    for(int i = 1; i <= plus_minus; i++){    //покроково додаємо елементи
        enter_plus:
        cout << "Введіть новий елемент масиву A: ";
        cin >> *(array_A + n_moment);
        if(cin.fail()){
            cout << "Ви ввели недопустиме значення. Спробуйте ще раз." << endl;
            cin.clear();
            cin.ignore(10000, '\n');
            goto enter_plus;
        }
        cout << "Додано: " << *(array_A + n_moment)  << endl;
        n_moment++;
        cout << "A = { ";
        for(int j =0; j <n_moment; j++){
            cout << *(array_A + j) << " ";
        }
        cout << "}" << endl;
    }
    plus_minus = 0;
```

```
    minus:
    cout << "Скільки ви хочете відняти елементів: "; //вводимо скільки хочемо відняти
    cin >> plus_minus;
    if(plus_minus > n_moment){
        cout << "Ви хочете відняти більше ніж на даний момент є. Віднімемо всі
елементи" << endl;
    }
    if(cin.fail() || plus_minus < 0){
        cout << "Ви ввели недопустиме значення. Спробуйте ще раз." << endl;
        cin.clear();
        cin.ignore(10000, '\n');
        goto minus;
    }
    if(plus_minus == 0){
        cout << "A = { ";
        for(int j =0; j <n_moment; j++){
            cout << *(array_A + j) << " ";
        }
        cout << "}" << endl;
    }


    for(int i = 1; i <= plus_minus; i++){    //покроково віднімаємо елементи
        if(n_moment == 0) break;
        cout << "Знято: " << *(array_A + n_moment - 1)  << endl;
        n_moment--;
        cout << "A = { ";
        for(int j =0; j <n_moment; j++){
            cout << *(array_A + j) << " ";
        }
        cout << "}" << endl;
    }


    choice:
    cout << "Хочете ще раз? (y/n): ";
    cin >>  choice;

    switch(choice){

        case 'y':
        goto start;
        break;

        case 'n':
        return 0;
```

```
        default:
        cout << "Не припустиме значення, спробуйте ще раз" << endl;
        cin.clear();
        cin.ignore(10000, '\n');
        goto choice;

    }

    return 0;
}
```

Код 1 – Lab 4

Посилання на файл коду в пулреквесті: посилання

Завдання №4 Lab# programming: Lab 5

```
#include <iostream>
using namespace std;

void arithmetic_mean(int n, int a[][4], float *sum, int column){
    for(int i =0; i <n; i++){
        *sum += a[i][column -1];
    }
    *sum /= n;
    cout << "Середнє арифметичне " << column << "-го стовпця дорівнює " << *sum <<
endl;
}

void arithmetic_much(float *sum, int n, int a[][4]){
    int number = 0;
    for(int i =1; i <4; i++){
        arithmetic_mean(n, a, sum, i);
        for(int j =0; j <n; j++){
            if(a[j][i] > *sum){
                number++;
            }
        }
        cout << "У " << i+1 << " стовпці елементів, що перевищують середнє
арифметичне попереднього, є рівно " << number << endl;
        number = 0;
        cout << endl;
        *sum = 0;
    }
}
```

```
int main(){

    float sum_a =0; //середнє арифметичне заданого стовпця
    const int n = 5, k = 4;
    int a[n][k] = {
        { 4, 12,  5,  3},
        { 8,  1, 14, 20},
        { 7,  7,  2, 10},
        { 2,  9,  6, 13},
        { 4, 18, 10,  9}
    };
    arithmetic_much(&sum_a, n, a);

    return 0;
}
```

Код 2 – Lab 5

Посилання на файл коду в пулреквесті: посилання

Завдання №5 Lab# programming: Algotester Lab 2

```
#include <iostream>
using namespace std;


int main(){

    int N = 0;
    cin >> N;

    int r[N];
    for(int i =0; i <N; i++){
        cin >> r[i];
    }

    bool may_to_delete[N];
    for(int i =0; i <N; i++){
        may_to_delete[i] = true;
    }

    for(int i =0; i <N; i++){
        if(!(may_to_delete[i])) continue;
        for(int j =1; j <(N-i); j++){
            if(r[i] == r[i+j]){
                may_to_delete[i] = false;
                may_to_delete[i+j] = false;
            }
```

```
        }
    }


    int max = r[0], min = r[0];
    for(int i =0; i <N; i++){
        if(r[i] > max) max = r[i];
        if(r[i] < min) min = r[i];
    }
    int diff = max - min;


    for(int i =0; i <N; i++){
        if(!(may_to_delete[i])) continue;
        if(i != 1){
            max = r[1]; min = r[1];
        }else{
            max = r[0]; min = r[0];
        }
        for(int j =0; j <N; j++){
            if(j == i) continue;
            if(r[j] >= max) max = r[j];
            if(r[j] <= min) min = r[j];
        }
        if(max - min < diff) diff = max - min;
    }

    cout << diff;

    return 0;
}
```

Код 3 – Algotester Lab 2

Посилання на файл коду в пулреквесті: [посилання]

Завдання №6 Lab# programming: Algotester Lab 3

```
#include <iostream>
#include <string>
using namespace std;

int n_o_c(int n, int number, int m){
    for(int i = 1; i<=(number-m); i++){
        n /= 10;
    }
    n = n%10;
    return n;
}
```

```
int main(){

    string s;
    cin >> s;
    int length_copy = s.length();
    char s_copy[s.length()];

    int n = 0;
    int k = 0;
    int t = 0;
    int number_of_n = 0;
    int number_of_char = 0;
    for(int i =0; i < s.length(); i++){
        if(s[i] == s[i+1]){
            int j = i;
            while(s[j] == s[i]){
                n++;
                j++;
            }
            t = n;
            while(t != 0){
                number_of_n++;
                t /= 10;
            }
            s_copy[k] = s[i];
            for(int m =1; m <=number_of_n; m++){
                s_copy[k+m] = n_o_c(n, number_of_n, m) + '0';
            }
            length_copy -= n-(1+number_of_n);
            i += n-1;
            n = 0;
            k+= 1+number_of_n;
            number_of_n = 0;
        }else{
            s_copy[k] = s[i];
            k++;
        }
    }

    for(int i =0; i < length_copy; i++){
        cout << s_copy[i];
    }

    return 0;
}
```

Код 4 – Algotester Lab 3

Посилання на файл коду в пулреквесті: посилання

Завдання №7 Practice# programming: Class Practice Task

```
#include <iostream>
#include <iomanip>

void output(int **image, int rows, int cols){

    for(int i =0; i <rows; i++){
        for(int j =0; j <cols; j++){
            std::cout << *(*(image + i)+j) << " ";
        }
        std::cout << "\n";
    }
    std::cout << "\n";

}

void enter_rc(int &rows, int &cols, bool &fail){  //уводимо висоту та ширину
зображення

    std::cout << "Висота зображення: ";
    std::cin >> rows;
    if(std::cin.fail() || rows < 1 || rows > 20){
        std::cout << "Не припустиме значення. Вихід з програми.";
        fail = true;
        return;
    }

    std::cout << "Ширина зображення: ";
    std::cin >> cols;
    if(std::cin.fail() || cols < 1 || cols > 20){
        std::cout << "Не припустиме значення. Вихід з програми.";
        fail = true;
        return;
    }

}

void enter_image(int rows, int cols, int **image, bool &fail){ //вводимо яскравість
```

```
    std::cout << "Яскравість пікселів (0..9): " << std::endl;
    for(int i =0; i <rows; i++){
        for(int j =0; j <cols; j++){
            std::cin >> image[i][j];
            if(std::cin.fail() || image[i][j] < 0 || image[i][j] > 9){
                std::cout << "Не припустиме значення. Вихід з програми.";
                fail = true;
                return;
            }
        }
    }

}

void image_output(int rows, int cols, int **image){ //виводимо зображення

    //виводимо зображення у звичайному вигляді

    std::cout << "\nЗображення " << rows << " x " << cols << ":" << std::endl;
    output(image, rows, cols);

    //виводимо сам малюнок

    char palette[9] = {'.', ':', '-', '=', '+', '*', '#', '%', '@'};

    std::cout << "\nМалюнок:" << std::endl;
    for(int i =0; i <rows; i++){
        for(int j =0; j <cols; j++){
            if(*(*(image + i)+j) == 0) std::cout << "  ";
            else std::cout << palette[(*(*(image + i)+j)-1)] << " ";
        }
        std::cout << "\n";
    }
    std::cout << "\n";
}

void findExtremes(int **image, int rows, int cols, int &minRow, int &minCol, int
&maxRow, int &maxCol){ //статистика

    minRow = 0; minCol = 0; maxRow = 0; maxCol = 0;
    int max = image[0][0]; int min = image[0][0];
    for(int i =0; i < rows; i++){  //шукаємо максимальну(мінімальну) яскравість,
шукаємо рядки, стовпці з цими значеннями
        int* row = image[i];
        for(int j =0; j < cols; j++){
            if(*(row + j) > max){
                max = *(row + j);
                maxRow = i;
```

```
                maxCol = j;
            }
            if(*(row + j) < min){
                min = *(row + j);
                minRow = i;
                minCol = j;
            }
        }
    }

    std::cout << "Найтемніший піксель: " << min << " (рядок " << minRow + 1 << ",
стовпець " << minCol + 1 << ")" << std::endl;
    std::cout << "Найсвітліший піксель: " << max << " (рядок " << maxRow + 1 << ",
стовпець " << maxCol + 1 << ")" << std::endl;

    float sum_image = 0;
    float arithmetic_mean = 0;
    for(int i =0; i < rows; i++){  //шукаємо суму всіх пікселів
        int* row = image[i];
        for(int j =0; j < cols; j++){
            sum_image += *(row + j);
        }
    }
    arithmetic_mean = sum_image / (rows * cols);  //середня яскравість всього
зображення
    sum_image = 0;

    std::cout << "Середня яскравість: " << std::fixed << std::setprecision(2) <<
arithmetic_mean << std::endl << "Середня по рядках: ";
    float arithmetic_mean_rows = 0;
    for(int i =0; i < rows; i++){  //шукаємо суму пікселів за рядками
        int* row = image[i];
        for(int j =0; j < cols; j++){
            sum_image += *(row + j);
        }
        arithmetic_mean_rows = sum_image / cols; //середня яскравість за рядком
        std::cout << std::fixed << std::setprecision(1) << arithmetic_mean_rows << "
";
        sum_image = 0;
        arithmetic_mean_rows = 0;
    }
    std::cout << std::endl;

}

void invert_image(int rows, int cols, int **image){

    for(int i =0; i <rows; i++){
```

```
        for(int j =0; j <cols; j++){
            *(*(image + i) + j) = 9 - *(*(image + i) + j);
        }
    }

    std::cout << "\nЯскравість інвертовано." << std::endl;
    output(image, rows, cols);

}

void swapPixels(int *a, int *b){
    int t_t = 0;
    t_t = *a; *a = *b; *b = t_t;
}

void reflection(int **image, int rows, int cols){

    std::cout << "Зображення віддзеркалено." << std::endl << "Початкове:" <<
std::endl;
    output(image, rows, cols);

    int n_reflection = cols / 2;
    for(int i =0; i < rows; i++){
        for(int j =0; j < n_reflection; j++){
            swapPixels(&image[i][j], &image[i][cols - 1 - j]);
        }
    }

    std::cout << "Результат:" << std::endl;
    output(image, rows, cols);

}

int** rotateRight(int** img, int rows, int cols){

    int** image_right = new int*[cols];
    for(int i =0; i <cols; i++){
        image_right[i] = new int[rows];
    }

    for(int i =0; i <cols; i++){
        for(int j =0; j <rows; j++){
            *(*(image_right + i) + j) = *(*(img + (rows - 1 - j)) + i);
        }
    }

    for(int i =0; i < rows; i++){
    delete[] img[i];
```

```
    }
    delete[] img;
    img = nullptr;

    return image_right;

}

void Rotate_Output(int **&image, int &rows, int &cols){

    image = rotateRight(image, rows, cols);
    int time_t = 0;
    time_t = rows; rows = cols; cols = time_t;
    std::cout << "\nЗображення повернуто. Нові розміри: " << rows << " x " << cols <<
std::endl;
    output(image, rows, cols);

}

int main(){

    int rows = 0, cols = 0;   //висота та ширина зображення
    int minRow = 0, minCol = 0, maxRow = 0, maxCol = 0;
    bool fail = false;
    enter_rc(rows, cols, fail); if(fail) return 0;


    int **image = new int*[rows];  //масив(матриця) зображення(яскравості)
    for(int i = 0; i < rows; i++){
        image[i] = new int[cols];
    }
    enter_image(rows, cols, image, fail);
    if(fail){
        for(int i =0; i < rows; i++){
            delete[] image[i];
        }
        delete[] image;
        image = nullptr;
        return 0;
    }

    int choice;

    do{

        menu:
```

```
        std::cout << std::left << std::setw(18) << "\nПункт" << "Операція" <<
std::endl;
        std::cout << std::left << std::setw(12) << "1" << "Показати зображення" <<
std::endl;
        std::cout << std::left << std::setw(12) << "2" << "Статистика яскравості" <<
std::endl;
        std::cout << std::left << std::setw(12) << "3" << "Інвертувати яскравість" <<
std::endl;
        std::cout << std::left << std::setw(12) << "4" << "Віддзеркалити по
горизонталі" << std::endl;
        std::cout << std::left << std::setw(12) << "5" << "Повернути на 90 градусів
праворуч" << std::endl;
        std::cout << std::left << std::setw(12) << "0" << "Вийти" << std::endl;

        std::cout << "\n>";
        std::cin >> choice;
        if(std::cin.fail() || choice < 0 || choice > 5){
            std::cout << "Ви ввели недопустиме значення, спробуйте ще раз." <<
std::endl;
            std::cin.clear();
            std::cin.ignore(10000, '\n');
            goto menu;
        }
        std::cin.ignore(10000, '\n');

        switch(choice){

            case 1:
            image_output(rows, cols, image);
            break;

            case 2:
            findExtremes(image, rows, cols, minRow, minCol, maxRow, maxCol);
            break;

            case 3:
            invert_image(rows, cols, image);
            break;

            case 4:
            reflection(image, rows, cols);
            break;

            case 5:
            Rotate_Output(image, rows, cols);
            break;

            default:
```

```
            std::cout << "Вихід з програми.";
            for (int i =0; i < rows; i++){
                delete[] image[i];
            }
            delete[] image;
            image = nullptr;
            return 0;

        }


    }while(choice != 0);

    for (int i =0; i < rows; i++){
        delete[] image[i];
    }
    delete[] image;
    image = nullptr;


    return 0;
}
```

Код 5 – Class Practice Task: редактор монохромного зображення

Посилання на файл коду в пулреквесті: посилання

Завдання №8 Practice# programming: Self Practice Task 1

```
/*
Белаш Матвій ШІ-12
Завдання з алготестеру "Допоможе чи заб'є?"
*/
#include <iostream>
#include <string>
using namespace std;

int main(){

    int k = 0;
    cin >> k;

    string s;
    cin >> s;

    int a = 0;
    int size = s.length();
```

```
    for(int i = 0; i < size; i++){
        if(size - i < 6){
            break;
        }
        if(s[i] == 'T'){
            if(s[i+1] == 'O'){
                if(s[i+2] == 'I'){
                    if(s[i+3] == 'L'){
                        if(s[i+4] == 'E'){
                            if(s[i+5] == 'T'){
                                a ++;
                                i += 5;
                            }
                        }
                    }
                }
            }
        }
    }

    if(a >= k) cout << "YES";
    else cout << "NO";

    return 0;
}
```

Код 6 – Self Practice Task 1

Посилання на файл коду в пулреквесті: [посилання]

Завдання №9 Practice# programming: Self Practice Task 2

```
/*
Белаш Матвій ШІ-12
Завдання з алготестеру "Спекотні дні пінгвінів"
*/
#include <iostream>
using namespace std;

int main(){

    int l=0, w=0, u=0, d=0;
    cin >> l>> w >> u >> d;
    if(l > w || l > (u+d)){
        cout << "Forget about the cocktails, man!";
        return 0;
    }else{
```

```
        cout << "Three times Sex on the Beach, please!";
        return 0;
    }
}
```

Код 7 – Self Practice Task 2

Посилання на файл коду в пулреквесті: посилання

Завдання №10 Practice# programming: Self Practice Task 3

```
/*
Белаш Матвій ШІ-12
Завдання з алготестеру "Літня школа"
*/
#include <iostream>
using namespace std;

int main(){

    int n=0, k=0;
    cin >> n >> k;

    if(n>(k*3) || n < k){
        cout << "Impossible";
        return 0;
    }

    int m = 0;

    if(n % k == 0){
        m = n / k;
    }else m = (n/k) + 1;

    int a[k];
    for(int i=0; i<k; i++){
        a[i] = 0;
    }
    for(int i=1; i<=m; i++){
        for(int j=0; j<k; j++){
            a[j]++;
            n--;
            if(n==0){
```

```
                for(int u=0; u<k; u++){
                    cout << a[u] << " ";
                }
                return 0;
            }
        }
    }
}
```

Код 8 – Self Practice Task 3

Посилання на файл коду в пулреквесті: посилання

Завдання №11 Practice# programming: Self Practice Task 4

```
/*
Белаш Матвій ШІ-12
Завдання з алготестеру "Повний чіназес"
*/
#include <iostream>
#include <string>
using namespace std;


int main(){

    int n = 0;
    int balance = 0;
    cin >> n;
    string N[n];
    int z = 0;
    bool dolg = false;
    for(int i =0; i < n; i++){
        cin >> N[i];
        cin >> z;
        if(N[i] == "earn"){
            balance += z;
        }
        if(N[i] == "spend"){
            balance -= z;
        }
        if(balance < 0){
            dolg = true;
        }
    }
```

```
    if(dolg == true){
        cout << "debt";
        return 0;
    }

    cout << "chinazes";
    return 0;
}
```

Код 9 – Self Practice Task 4

Посилання на файл коду в пулреквесті: посилання

5. Результати виконання завдань, тестування та фактично затрачений час:

Завдання №3 Lab# programming: Lab 4

```
0/100 елементів масиву
Скільки ви хочете додати елементів: 3
Введіть новий елемент масиву A: 2
Додано: 2
A = { 2 }
Введіть новий елемент масиву A: 4
Додано: 4
A = { 2 4 }
Введіть новий елемент масиву A: 1
Додано: 1
A = { 2 4 1 }
Скільки ви хочете відняти елементів: 2
Знято: 1
A = { 2 4 }
Знято: 4
A = { 2 }
Хочете ще раз? (y/n): n
```

Рисунок 24 – Результат роботи Lab 4

Моя програма пропонує виконувати ці дії стільки, скільки захоче користувач. Якщо користувач уведе завелике значення, або не коректну відповідь, то програма зарахує максимальну кількість елементів, якщо значення, введене користувачем завелике, та попросить ввести значення ще раз, якщо воно не коректне.

Час затрачений на виконання завдання: 30 хв

Завдання №4 Lab# programming: Lab 5

```
Середнє арифметичне 1-го стовпця дорівнює 5
У 2 стовпці елементів, що перевищують середнє арифметичне попереднього, є рівно 4

Середнє арифметичне 2-го стовпця дорівнює 9.4
У 3 стовпці елементів, що перевищують середнє арифметичне попереднього, є рівно 2

Середнє арифметичне 3-го стовпця дорівнює 7.4
У 4 стовпці елементів, що перевищують середнє арифметичне попереднього, є рівно 4
```

Рисунок 25 – Результат роботи Lab 5

Програма виводить значення, які випливають зі змінних заданих у коді. Так як у вимогах не вказано про уведення цих змінних користувачем, я ініціалізував ці змінні у коді. Однак перевірити правильність написання коду можна, помінявши змінні у коді.

Час затрачений на виконання завдання: 30 хв

Завдання №5 Lab# programming: Algotester Lab 2

```
5
1 2 2 4 4
2[1] + Done
```

Рисунок 26 – Результат тестування Algotester Lab 2

Час затрачений на виконання завдання: 30-40 хв

Завдання №6 Lab# programming: Algotester Lab 3

```
AAAABBBCQQQQ
A4B3CQ4[1] + Done
```

Рисунок 27 – Результат тестування Algotester Lab 3

Час затрачений на виконання завдання: 1 година

Завдання №7 Practice# programming: Class Practice Task

```
Висота зображення: 4
Ширина зображення: 5
Яскравість пікселів (0..9):
0 1 3 6 9
1 2 4 7 9
2 4 6 8 9
3 5 7 9 9

Пункт       Операція
1           Показати зображення
2           Статистика яскравості
3           Інвертувати яскравість
4           Віддзеркалити по горизонталі
5           Повернути на 90 градусів праворуч
0           Вийти

>1

Зображення 4 x 5:
0 1 3 6 9
1 2 4 7 9
2 4 6 8 9
3 5 7 9 9


Малюнок:
  . - * @
. : = # @
: = * % @
- + # @ @
```

Рисунок 28 – Class Practice Task: введення зображення та пункт 1 меню («Показати зображення»)

```
>2
Найтемніший піксель: 0 (рядок 1, стовпець 1)
Найсвітліший піксель: 9 (рядок 1, стовпець 5)
Середня яскравість: 5.20
Середня по рядках: 3.8 4.6 5.8 6.6
```

Рисунок 29 – Class Practice Task: пункт 2 меню («Статистика яскравості»)

```
>3

Яскравість інвертовано.
9 8 6 3 0
8 7 5 2 0
7 5 3 1 0
6 4 2 0 0
```

Рисунок 30 – Class Practice Task: пункт 3 меню («Інвертувати яскравість»)

```
>4
Зображення віддзеркалено.
Початкове:
0 1 3 6 9
1 2 4 7 9
2 4 6 8 9
3 5 7 9 9

Результат:
9 6 3 1 0
9 7 4 2 1
9 8 6 4 2
9 9 7 5 3
```

Рисунок 31 – Class Practice Task: пункт 4 меню («Віддзеркалити по горизонталі»)

```
>5

Зображення повернуто. Нові розміри: 5 x 4
3 2 1 0
5 4 2 1
7 6 4 3
9 8 7 6
9 9 9 9
```

Рисунок 32 – Class Practice Task: пункт 5 меню («Повернути на 90 градусів праворуч»)

Час затрачений на виконання завдання: 4-5 годин

Завдання №8 Practice# programming: Self Practice Task 1

0081 - Допоможе чи заб'є?	C++	Зараховано	0.005	1.203	2324094

Рисунок 33 – Результат перевірки в Алготестері: «Допоможе чи заб'є?»

Час затрачений на виконання завдання: 20-30 хв

Завдання №9 Practice# programming: Self Practice Task 2



Рисунок 34 – Результат перевірки в Алготестері: «Спекотні дні пінгвінів»

Час затрачений на виконання завдання: 5-10 хв

Завдання №10 Practice# programming: Self Practice Task 3



Рисунок 35 – Результат перевірки в Алготестері: «Літня школа»

Час затрачений на виконання завдання: 30 хв

Завдання №11 Practice# programming: Self Practice Task 4



Рисунок 36 – Результат перевірки в Алготестері: «Повний чіназес»

Час затрачений на виконання завдання: 40 хв

6. Кооперація з командою:



Рисунок 37 – Командна робота над Епіком 4

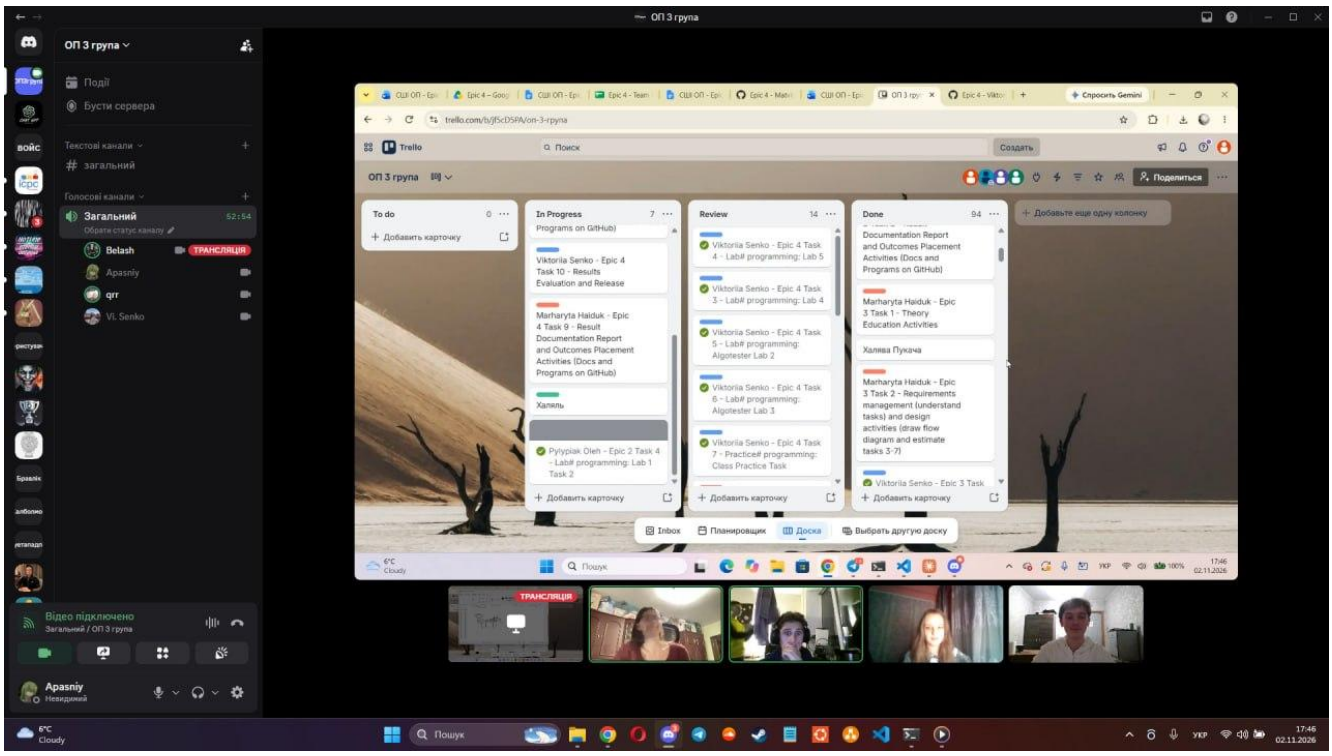


Рисунок 38 – Командна робота над Епіком 4 (обговорення всіх завдань)

Висновки:

Під час виконання цього Епіка я вивчив відносно нові для себе речі: двовимірні масиви, вказівники та посилання. Мені довелось з цим попрацювати та закріпити в голові. Я не думаю, що знаю це на 100%. Однак в основному, базові і не тільки речі зробити зможу. Нам казали, що багато що з цього Епіка в подальшому не знадобиться в таких масштабах, максимум трошки. Проте я думаю, що це загалом був доволі хороший досвід, тому що я, як мінімум, ще краще освоїв старі та нові навички.